

Grok Bot Technical Report

Subject: xAI / SpaceXAI Grok Bot (<https://x.ai/bot>)

Scope: Documented capabilities, construction model, identity system, connectors/MCP, observed use cases, and a replication architecture

As of: 5 September 2026

Method: Official product and docs pages, launch posts, security FAQ, and independently reported user fleets. Marketing claims are labeled separately from documented constraints.

1. What Grok Bot actually is

Grok Bot is not the conversational Grok assistant at grok.com, not X's [@grok](#) account, and not Grok Build (the terminal coding agent). Official docs define a Bot as:

a single persistent, named agent or one AI teammate.

Product positioning: “AI teammates you can give real work to.” Each account gets a persistent cloud computer. Bots sign into tools the way a person would, finish work in the real app, and only return when approval is needed.

The product launched 11 August 2026 in early beta. The legal entity on current pages is SpaceXAI LLC; the product still lives at x.ai/bot and is implemented as a Cursor-hosted agent runtime. Sign-in is a Cursor account. The desktop installer is served from Cursor update infrastructure; onboarding routes through Cursor. Privacy, billing, SSO, and much of the plugin policy inherit Cursor, not a standalone xAI control plane.

That hosting split is the most important architectural fact. Grok Bot is a computer-use agent product with a chat UX, shipped as a thin client over a Cursor cloud VM.

2. Documented and observable capabilities

2.1 Core runtime

Capability	What docs say	Practical implication

Persistent cloud VM	Browser + filesystem + terminal, survives laptop close	Work is not a chat draft; it lands in real tools
Shared computer	One computer per user, not per Bot	Logins, cookies, /workspace files, CLI creds are shared
Parallel Bots	Multiple Bots run at once; each has its own screen	Screens are work surfaces, not security boundaries
Computer-use concurrency	One computer-use task per Bot screen at a time	Parallelism is across Bots, not infinite tabs on one Bot
Background / 24x7	Cloud computer continues when the client is closed	Routines and long turns are server-side
Connectors / plugins	Structured APIs when available	Preferred over clicking a site
Computer use fallback	Browser for tools with no API/MCP	Covers “messy SaaS” and visual workflows
Multi-Bot messaging	DMs, group chats (2–6 Bots), @ mentions, ownership handoff	User is not required to be the router
Skills	Reusable how-to with validation and approval rules	Cross-Bot playbooks
Routines	Schedule or (where supported) event trigger	Recurring jobs without a workflow builder
Teach-by-demo	Record up to 10 minutes of browser work	Draft skill, not a finished automation
Memory	Role context, preferences, summaries of prior work	Conversation memory is per Bot; files/sessions are shared
Approvals / Auto-review	Allow once / deny / always allow; team rules	HITL on send, spend, delete, publish, prod change
Human takeover	User types password, 2FA, CAPTCHA on the VM	Secrets should not go in chat
Sharing	Public config link copies identity/skills/routines	Does not share computer, logins, or history

Sources for the table: overview, computer-and-apps, FAQ, skills/routines, chat, bots.

2.2 Clients and platforms

Thin clients exist for:

platforms exist for:

- macOS (Apple silicon and Intel)
- Windows (x64 and Arm64)
- Linux (x64 and Arm64: .deb , .rpm , AppImage)
- iPhone (iOS 18+)
- Android 9+

iPad was not supported at launch. Mobile can start work, approve, watch the computer, and manage routine on/off; schedule editing and run history still require desktop.

2.3 Access and billing (documented, plan names have drifted)

Docs currently list eligibility as SuperGrok Plus, SuperGrok Heavy, Cursor Pro+, Cursor Ultra, Cursor Teams Standard/Premium. Launch-day copy was narrower (Heavy / Ultra / Teams Premium). Grok Bot usage is separate from ordinary Grok/Cursor chat quotas, with weekly included usage and on-demand token billing. If both Cursor and SuperGrok subscriptions exist, Grok Bot uses whichever has more usage. Enterprise is still rolling out; several admin controls are Enterprise-only.

2.4 What it is *not*

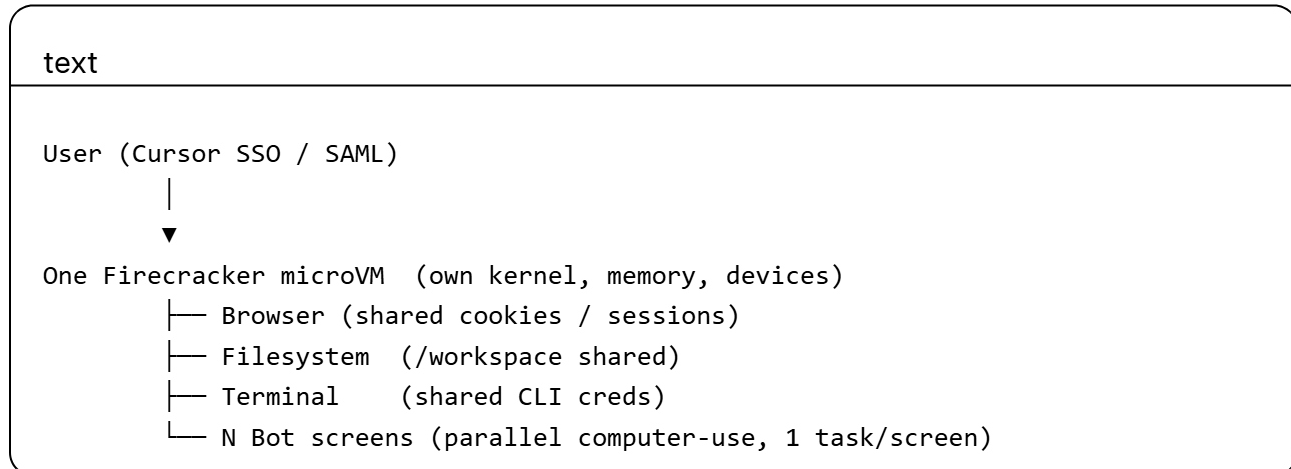
Documented negatives matter as much as features:

- No on-prem, no BYO image, no deploy-inside-your-VPC. Computers run in the United States on Cursor-hosted Firecracker microVMs.
- Legacy Privacy Mode is incompatible; cloud storage is required.
- Bots have no independent machine identity. They act as the signed-in member. Team-managed connectors are the exception.
- Local MCP/stdio servers on the user's laptop are not reachable. The agent lives in the cloud. Third-party writeups and Cursor staff comments match this.
- Blocking a plugin does not block the same vendor's website in the browser. Closing both paths needs connector policy and Enterprise Network Controls.
- Isolation is per user, not per Bot. Separate Bots are not a security boundary. For a separate credential set, docs say: give the workload its own Cursor user.

3. Construction model

Grok Bot is assembled from six layers. There is no visual workflow builder. Construction is: name a teammate → give it a job → grant tools → run a task → freeze the method as a skill → optionally schedule it.

3.1 Layer A — Account and computer



User-to-user isolation is strong (Firecracker). Bot-to-Bot isolation inside one user is intentionally weak so handoffs work without re-login.

3.2 Layer B — Named Bot object

Create path: New → Create new agent → default name “New Agent” → Edit Profile.

Profile fields:

- Name
- Title
- Description (standing instructions / policy)
- Avatar

Account cap: 50 Bots + group chats combined. Duplicate copies profile, settings, enabled skills, routines, avatar — not history, learned memory, or attachments. Delete removes profile, conversation, and routines; shared-computer files and sessions remain.

3.3 Layer C — Task loop

A strong first request is specified as five fields: outcome, sources, constraints, deliverable, review point. The transcript mixes chat, tool activity, computer-use, files, questions, and approval cards. A later user message can redirect an in-flight turn; “Stop now” halts new work but does not undo completed actions

work but does not undo completed actions.

3.4 Layer D — Skills

A skill is a reusable procedure, not a compiled program. Official structure:

1. When to use it
2. Required inputs and access
3. Sequence of work
4. How to validate the result
5. What to return
6. What requires approval

Skills can be written after a successful run, installed from Settings → Plugins, or drafted from a “Teach a task” recording (≤10 minutes, no mic audio, secrets discouraged). Skills are account-available; a given Bot still needs the connector/login. Private skills can be enabled per Bot. Invoke with / .

3.5 Layer E — Routines

A routine binds one Bot + one workflow + a schedule or event. Example pattern from docs: weekday 08:00, run skill X against list Y, post watchlist, never contact customers, fail closed if source data is missing.

Limits: 50 routines per Bot; 20 most recent run records kept. Missing the user for a long period can pause routines. Event triggers should be narrow (“message in #customer-escalations contains ticket link AND ‘needs repro’”), not “every new message.” Test before enable. Group threads reportedly cannot own routines in some observed setups; routines attach to a Bot.

3.6 Layer F — Multi-agent coordination

- 1:1 Bot conversations
- Group chats of 2–6 Bots
- @Bot , @group , @routine , @connector , @everyone
- Async handoff messages with a single owner per stage

Recommended roster pattern in docs: one Bot owns an end-to-end outcome; add a specialist only when the role is stable; use a group when the handoff itself must be visible. Example: Website Launch group = coordinator + content editor + analytics reviewer.

This is soft multi-agent: shared memory via files/sessions/messages, not a formal actor framework with isolated sandboxes.

4. Instruction, personality, and identity system

Grok Bot does not expose a public “system prompt API” or a companion-style character picker (Ani / Rudy / Valentine are a different Grok-app feature). Identity is operational, not theatrical.

4.1 Three instruction channels (documented split)

Channel	Persistence	Intended content
Profile description	Durable, copied on duplicate/share	Standing job, working style, hard rules (“Never send external messages without approval”)
Conversation messages	That Bot’s thread	Task-specific orders (“Draft follow-ups for these twelve accounts”)
Skills / routines / Auto-review	Durable, reusable	Procedure, schedule, action policy

Docs are explicit: use the description for rules that should remain true; use the message for this task.

4.2 What “personality” actually means here

Official examples are role + SOP + approval boundary, not tone cards:

Own the weekly account-health review. Pull product usage and support signals, flag evidence of churn or expansion, and produce a linked watch list for the customer-success team. Never contact a customer or change an account without approval.

Suggested starter roles on the marketing page: Sales Outbound, Talent Scout, Paid Media, Expense Manager, Product Performance, Bug Reproduction, Account Health, Chief of Staff.

Memory compounds per named Bot: preferences, role context, summaries. Docs warn not to treat memory as source of truth for important decisions — re-read the live system. Shared computer state (CRM session, /workspace files) is a second, stronger “memory” that every Bot can see.

4.3 Shareable identity

A public share link publishes configuration: identity, description, skills, routines. Recipients add a copy to their account. Secrets, customer data, and internal URLs must be stripped first. Adding a shared Bot accepts third-party bot terms. This is a template market, not a multi-tenant agent that keeps running on the author’s computer.

4.4 Team-level identity overlay

On Teams/Enterprise:

- Team Rules apply to every member’s Bots (always required, not optional).
- Auto Review team instructions add org-wide allow/block.
- Public template sharing can be locked to the team (Enterprise default: off).

There is still no first-class “Bot service account” in the IdP. The Bot is the human. To sign into IdP apps from the Linux VM, admins must add an auth rule that allows Linux / “Other Desktop” without device-trust agents. Plugin OAuth does not go through that computer browser path.

5. Connectors, integrations, and the three MCP surfaces

People collapse three different xAI products into one “Grok MCP.” They do not share a config file.

Surface	Product	How you attach tools	Transport
Grok chat connectors	grok.com / iOS / Android assistant	grok.com/connectors → catalog or Bring Your Own MCP	Public HTTP MCP; localhost rejected unless tunneled
Grok Build / CLI	Terminal coding agent	grok mcp add , ~/.grok/config.toml , local stdio and remote HTTP	Local + remote
Grok Bot	Cloud teammates	Settings → Plugins marketplace; packaged skills; custom MCP via plugin/ AddMcpServer in chat	Remote only from the cloud VM

This split is consistently reported by implementers and matches official docs for each surface.

5.1 Grok Bot tool preference order

Official guidance:

1. Prefer a connector/plugin when one exists (structured, more reliable).
2. Fall back to computer use for sites without an API/MCP or for visual workflows the connector does not expose.
3. Mentions: type @ to attach a connector to a task; / for a skill.

Installed connectors are account-wide, not scoped to one Bot.

5.2 Marketplace contents (observable, not a frozen official inventory)

The in-app catalog is large and moving. Independent inventories in late August / early September 2026 cite on the order of 200+ plugins, with a featured sales set of Apollo, Clay, Gong, HubSpot, plus commonly recommended Gmail, Slack, Notion, GitHub, Google Calendar, Firecrawl, Canva, Sentry, Composio (1,000+ apps behind one MCP broker).

X integration (29 August 2026): connect an X account; paid Grok Bot users get starter X API credits; Bots can search posts, read timeline, check mentions; there is also an X plugin for search, trends, bookmarks. First version, explicitly incomplete.

5.3 Custom MCP on Grok Bot — documented constraints

- Server must be reachable from Cursor’s cloud (public URL or private path via Enterprise Team Setup).
- Local stdio / localhost MCP is out of band for Grok Bot.
- Some writeups note `AddMcpServer` currently takes name + URL + optional headers, with weak support for OAuth client-id flows that Grok chat connectors handle better. Treat OAuth-heavy MCP as “verify on this surface” rather than assume parity.
- Enterprise can apply an MCP allowlist. Teams inherit Cursor connector policy; admins can restrict but cannot push mandatory connectors to members.

5.4 Private-network extensibility

Enterprise Team Setup installs shell manifests on every team computer (Debian-based Linux, sudo, no secrets in the manifest). Documented pattern: install Tailscale or cloudflare, authenticate in the computer browser, reach internal hostnames. Cursor provides the hook, not a managed overlay network. Network Controls must allow the client’s coordination endpoints. Regular traffic still uses Cursor shared egress unless

privately routed.

5.5 Adjacent xAI MCP (do not confuse)

- xAI API Remote MCP tools: `tools=[mcp(server_url=...)]` on grok-4.6, managed by xAI for API apps.
- Docs MCP: <https://docs.x.ai/api/mcp> for documentation retrieval.
- Grok chat BYO MCP: public URL + optional tunnel (ngrok etc.).

Useful for building *around* Grok Bot; they are not the Bot runtime.

6. Security and control plane (evidence, not brochure)

Isolation: per-user Firecracker microVM. Anysphere (Cursor) holds ISO/IEC 27001 and 42001; Grok Bot is in current ISO scope (`trust.cursor.com`).

Approvals: model-based Auto-review plus explicit cards. Require Approval beats Always Allow when both match. Approvals do not roll back work already done.

Secrets: takeover for passwords/2FA/CAPTCHA/payment; masked secret request for supported connections (value not added to transcript, not shown to the model). Docs: this is not a password manager. Never paste keys in chat.

Enterprise-only: org enable switch, Network Controls (allow-all / defaults+allowlist / allowlist-only, plus group lock), Action Recording (90-day internal store of scrubbed actions), OpenTelemetry export, audit log, SCIM 2.0, computer terminate (disk kept, next session is a fresh VM on that disk), MCP allowlist, Team Setup.

Hard limits for security reviewers: no DLP hooks; restricting egress is the substitute. Connector block $\neq$ website block. Team model allowlist “enforcement is not guaranteed.” No per-Bot sandbox. US hosting only.

7. Use cases with real examples

7.1 Vendor-stated internal jobs (launch post)

SpaceXAI published jobs already running internally: sales Bot updating CRM from call notes and drafting follow-ups; ops Bot seating new hires and processing Gmail invoices; engineering Bot reproducing a UI bug, filing a ticket, handing off to a debugging Bot; NDA-

collection sequences; overnight demo-environment repair; Monday pipeline scoreboards.

Canonical first-handoff prompt from docs (multi-system, draft-only):

Pull this week's Strategic Prospects PG List from Salesforce. Skip anyone already in a sequence. Research the top 5 accounts across the web, Slack, Databricks, and Sumble, pull contacts, and draft LinkedIn and email in my voice, and leave me drafts to approve by tomorrow morning.

The marketing Sales Outbound demo is the same shape: sheet → research → CRM context → 36 drafts overnight, zero sends.

7.2 Independently documented user fleets

These are named setups with public walkthroughs, not hypothetical personas.

- Brandon Charleson — ~15-bot business fleet. “Eve” as chief of staff designs the roster and produces daily roll-ups. “Postmaster” classifies inboxes and routes money events. “Talia” (CFO) reconciles invoices/subscriptions to bank statements and recommends keep/cut. Human still approves sends and payments.
- Peter Yang — advisor-spawned five-pack. An Advisor Bot proposes specialists after a briefing: YouTube researcher (daily briefs), X Scout (weekly top posts), “Digital Marie Kondo” (Gmail/Drive audit with no delete/unsubscribe).
- Nate B Jones — 12 themed bots under a Chief of Staff. Emphasis on outcome-owned specialists (e.g., schedule optimizer that only *suggests* slots) rather than a generic inbox bot.
- Riley Brown — EA digest Bot. 09:00 roll-up from Linear, Slack, email, calendar; sponsorship mail prioritized; report-only.
- Clearmud — X growth trio. Researcher + Creative + Drafter with independent routines. Observed snag: routines not savable from group threads.
- Paul J Lipsky — Atlas + Lincoln. Travel monitoring that created its own fare/hotel routine; LinkedIn research filed to Notion via screen takeover so credentials never sat in chat.

7.3 Pattern that actually works

Across vendor and user examples, successful Bots share five traits:

1. One job, one approval boundary, one source-of-truth system.
2. Draft / reconcile / recommend first; send / spend / delete last.

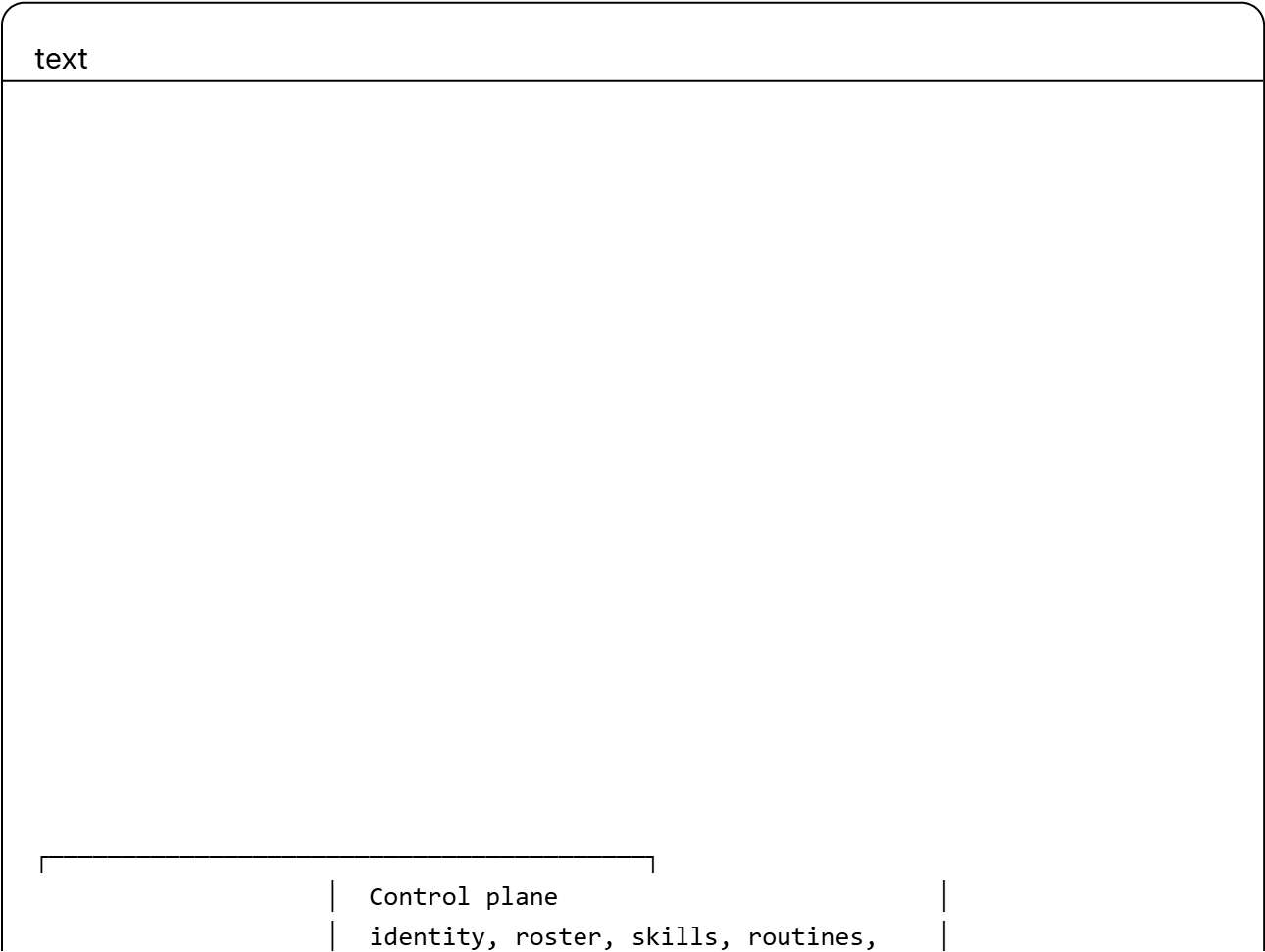
- 3. A Chief of Staff Bot for routing, not 20 peer Bots DMing the human.
- 4. Skills written after a clean manual run, then scheduled.
- 5. Connector used when it exists; browser used when the UI *is* the work (bug repro, portal with no API).

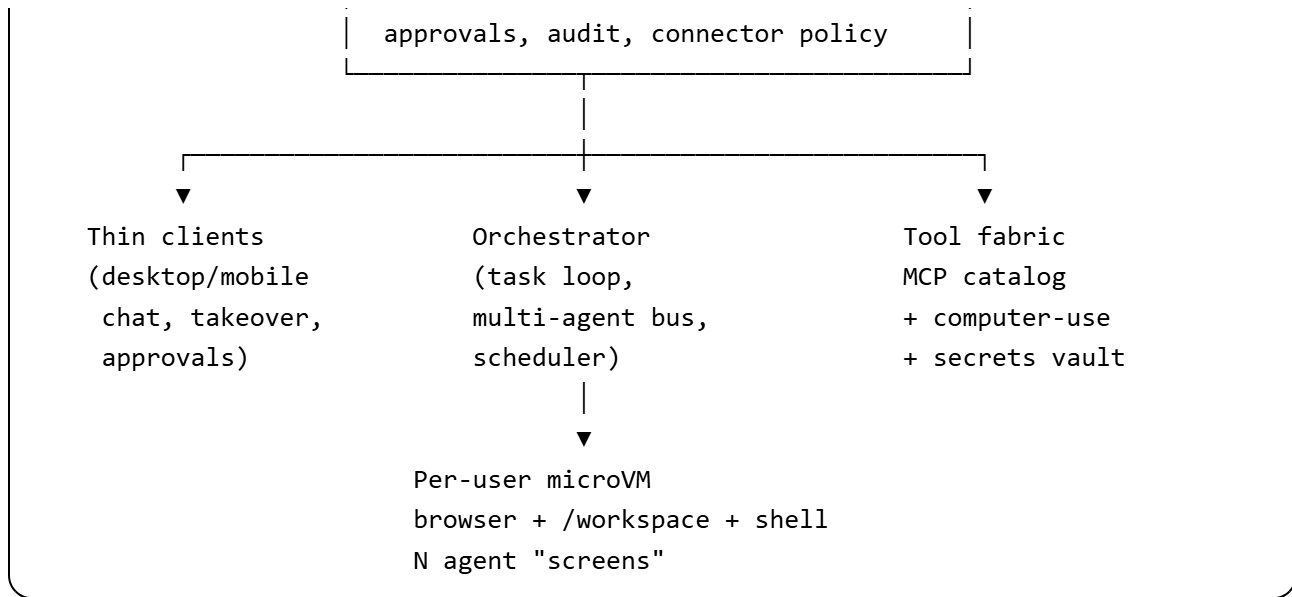
Failure modes already visible in the wild: catch-all “General Helper” Bots with mushy memory; treating Bot split as isolation; event triggers that are too broad and burn weekly usage; assuming local MCP or device-trust SSO will just work; sharing a Bot link that still contains internal URLs.

8. Replication architecture

You cannot legally or practically clone the hosted product. You *can* replicate the architecture with open components. Below is a faithful mapping of Grok Bot’s documented design, not a Cursor/xAI reimplementation.

8.1 Target architecture





8.2 Component mapping

Grok Bot primitive	Replication choice	Notes
Firecracker microVM	Firecracker or gVisor + persistent disk	One VM per user if you want their isolation model
Shared /workspace	Network volume mounted at /workspace	Project folders as the only durable store
Computer use	Browser-use / Playwright / Chromium CDP with screenshot+a11y tree	One active GUI task per agent screen
Named Bot	Row: {id, name, title, description, avatar, memory}	Description = standing policy
Skills	SKILL.md (when, inputs, steps, validate, return, approval)	Same six-part schema as docs
Teach-a-task	Record CDP events 10 min → LLM compile to SKILL.md draft	Always require human edit
Routines	Cron + optional webhook matcher	Fail closed on missing source data
Multi-agent	Shared thread store + @ router; cap group size at 6	Single owner per stage
Plugins	MCP servers, namespaced server__tool	Prefer MCP; GUI fallback

Custom MCP	Remote Streamable HTTP/SSE only from the VM	No laptop stdio
Approvals	Policy engine: description rules $\cup$ org rules $\cup$ action class	Require-approval wins
Secrets	Separate vault; masked tool; human takeover channel	Never in transcript
Identity	Human IdP only; optional team connector SAs	No extra machine principal unless you choose otherwise
Private net	Tailscale/Cloudflare on the VM via startup manifest	Secrets out of the manifest
Clients	Chat app + live view of the VM display	Mobile = approve + glance

8.3 Agent loop (minimal spec)

text

```
on message(user or peer or schedule):
  load Bot.profile + Bot.memory + attached skills + allowed tools
  plan against outcome/sources/constraints/deliverable/review_point
  while not done:
    if action in {send, publish, pay, delete, prod_write}:
      request_approval(); wait
    if tool has MCP: call MCP
    else if needs GUI: computer_use on this Bot's screen
    if needs secret: pause → human takeover
    write artifacts to /workspace/<project>/
    emit action log + screenshots/links
  update memory summaries
  if peer_handoff: post to target Bot with owner + artifact paths
```

8.4 Identity pack (the actual “personality system”)

Store four artifacts per Bot, not one mega-prompt:

1. Charter — job, systems in scope, out of scope, approval line.
2. Style card — voice, artifact format, how to cite evidence.
3. Skills index — enabled SKILL.md ids.
4. Memory — durable facts vs “read-only source” flags

4. memory — durable facts vs. re-read source files.

Grok Bot's documented quality comes from narrow charters, not from a clever persona.

8.5 Roadmap to a working replica

Phase 0 — Decide isolation.

Copy their model (shared VM per human) only if handoff convenience beats blast radius.
For agencies or multi-client work, break their model: one VM per client.

Phase 1 — Single agent, no GUI (2–3 weeks).

Chat + remote MCP + `/workspace` + approval cards + SKILL.md. Prove inbox-digest and CRM-draft jobs.

Phase 2 — Computer use (3–5 weeks).

Persistent Chromium profile, takeover UI for 2FA, one-task-per-screen lock, session sharing policy.

Phase 3 — Roster and bus (2–3 weeks).

Named agents, 2–6 group threads, @ router, Chief-of-Staff pattern, 50-agent cap if you want operational similarity.

Phase 4 — Skills and routines (2 weeks).

Compile from successful traces; scheduler; 20-run history; stale-data policy.

Phase 5 — Teach-by-demo (2–4 weeks).

Record, summarize, human-edit, test on a safe fixture before scheduling.

Phase 6 — Control plane (ongoing).

SSO, connector allowlist, egress policy, action recording + OTel, SCIM. This is where Cursor's Enterprise surface lives; it is most of the product's institutional value and little of the demo.

Phase 7 — Do not bother cloning.

US-only hosting story, public Bot template links, Cursor billing bundling, X API credit grants. Those are distribution, not architecture.

8.6 Where a replica will lag

- Reliability of computer use on hostile websites (CAPTCHA, bot shields). Grok Bot's own FAQ admits sites can block automation.
- Connector depth vs. a 200-plugin marketplace.
- Memory quality across weeks of messy work.
- Team SSO + Linux VM + device-trust exceptions — a surprising amount of real-world friction.
- Evaluation: they sell “finished work,” which is harder to unit-test than chat.

A replica that only does MCP + cron + HITL already covers a large fraction of the *successful* public fleets, because those fleets deliberately stop at drafts.

9. Implications

For individuals. Grok Bot is closest to “hire a remote generalist with one shared laptop.” The shared-computer choice is why fleets feel fast and why you should not put a production admin session next to an experimental scraping Bot.

For teams. The interesting surface is not the cute roster. It is Firecracker + Cursor SSO + plugin policy + (Enterprise) egress allowlists + action recording. Self-serve Teams get the agent, not the security product.

For builders. Extensibility is MCP-shaped, but Grok Bot is the least local of the three Grok MCP clients. Ship a hosted Streamable HTTP server, package a plugin (skills + MCP), and assume computer use as backup. Do not design for stdio-on-the-laptop.

For anyone comparing “custom Grok agents.” Chat-app custom agents, Grok companions, Grok Build plugins, and Grok Bot teammates are four products. Only Grok Bot gives a persistent cloud computer and 24×7 routines as a first-class object.

10. Primary sources

Official: x.ai/bot, [Introducing Grok Bot](https://x.ai/grok-bot), [Grok Bot + X](https://x.ai/grok-bot+X), docs under docs.x.ai/grok-bot/ (overview. get-started. bots. chat-and-collaboration. computer-and-apps. skills-routines-

and-automations, approvals-security-and-privacy, security, security-faq, identity-and-access, teams-and-enterprises, private-networks, mobile, faq). Adjacent MCP docs live under Grok connectors, Grok Build MCP, and the xAI API remote-MCP tools pages.

Observed fleets and plugin inventories are secondary and dated; treat catalog counts as snapshots.

If you want a follow-up, the highest-leverage next artifacts are: (1) a Bot charter template library mapped to the official six-part skill schema, or (2) a concrete replication repo skeleton (control-plane schema + Firecracker userdata + MCP gateway + approval worker) aimed at one vertical such as sales-outbound or engineering-renro